

LinkedIn Post Topics

PersonalFinance API — Architecture, Patterns, Security, Observability, AWS & AI-Assisted Dev

ARCHITECTURE

#1

Building a Modular Monolith in .NET 8

Why I chose a modular monolith over microservices for a solo project — team autonomy without deployment complexity. Each module (Account, User, Credentials) is fully self-contained with its own handlers, repositories, and projections, but shares a single process.

#2

Hand-Rolling CQRS Without MediatR

CQRS from scratch: a dispatcher pattern with just interfaces and DI service resolution. Why I skipped MediatR — what you gain (zero magic, explicit control), what you lose (pipeline behaviors), and whether the tradeoff was worth it.

#3

Event Sourcing for Financial Accounts

Every balance change, payment, and credit adjustment is a permanent, immutable event. No UPDATE statements on the Account table — only appends to the Event Store. What this gives you: a full audit trail, time-travel queries, and aggregate rebuilds from scratch.

#4

The AggregateRoot Pattern in Practice

How my base class tracks CurrentVersion/NextVersion for optimistic concurrency, records uncommitted events, and forces every state change through Apply(). Concrete example with the Account aggregate.

#5

Optimistic Concurrency Without EF Core

Raw ADO.NET + UPDLOCK + ROWLOCK hints to prevent lost updates in a high-write financial system. How event version checking replaces the usual RowVersion column approach.

#6

One Aggregate, One Stream — Event Store Keys Done Right

The stream key must be the aggregate's own identity, never a foreign key. Why keying UserCredentials events by UserId silently mixed two aggregates (User + Credentials) into one stream — causing replay crashes and version collisions — and how moving to the aggregate's own Id untangled it. A foreign key is a query concern, not a storage key.

PATTERNS & DESIGN

#7

The Transactional Outbox Pattern — Why It Exists and How I Built It

Appending to the Event Store and the Outbox in a single SQL transaction — then a BackgroundService polls and dispatches to projections. The problem it solves: no event is ever lost even if the app crashes between persist and publish.

#8

Projections and Read Models: CQRS in the Database Layer

Separate write-side (Event Store) and read-side (projection tables) in the same SQL Server. Projections subscribe to domain events and build query-optimized views. ProjectionCheckpoint for idempotency.

#9

DomainResult<T> — Returning Errors Without Throwing Exceptions

Why domain validation failures in this project are raised as events (ApplyDeltaErrorEvent), not exceptions. Command handlers inspect uncommitted events for errors before persisting. The result: clean error propagation without try/catch in the domain layer.

#10

The Money Value Object

A readonly record struct that enforces 3-letter ISO currency codes, rounds to 2 decimal places with MidpointRounding.AwayFromZero, and blocks cross-currency arithmetic at compile time. Why this type exists and what bugs it prevents.

#11

No ORM: Raw ADO.NET in a Production System

IDbConnectionFactory, ICommandFactory, parameterized queries, SqlDataReaderExtensions. What you gain: full SQL control, no N+1 surprises, no migration headaches. What you give up: convenience. Worth it for financial data.

SECURITY

#12

Two-Stage JWT Authentication with 2FA

The partial/full token split: after password login you get a short-lived partial token (5 min). Only after TOTP verification do you receive a full token with account claims — delivered as HttpOnly, SameSite=Strict cookies, never in the response body. How this design prevents 2FA bypass and keeps the token away from JavaScript.

#13

Encrypting JWT Claims with AES

User ID, email, and account IDs are AES-encrypted inside the JWT payload. Even if the token is decoded, sensitive claims are opaque. How ClaimEncryption works and why the token body is still signed.

#14

One Decryptor to Rule Them All — Centralizing JWT Claim Handling

Five controllers each carried twelve lines of claim-decryption boilerplate — config lookup, AES decrypt, Guid parsing, error mapping. A single injectable JwtClaimsDecryptor now decrypts the whole token (user id, login, account ids) in one call and maps failures to proper 401s. Less code, no crypto-exception 500s, and the decrypted AccountIds enabled per-account ownership checks.

#15

Enforcing 2FA at the Framework Door with Authorization Policies

A one-line policy — RequireClaim("token_type", "full") — applied class-level rejects pre-2FA partial tokens before any controller code runs. The subtlety: attribute policies stack, so the verify-2FA endpoint (which needs a partial token) forces per-action policies on its controller. Framework-level gates beat copy-pasted token checks.

#16

TOTP 2FA from Scratch with OtpNet and QRCode

Implementing Time-based One-Time Passwords: generating a 20-byte base32 secret, building an otpauth:// URI, rendering a QR code as base64, and verifying codes with a +/-1 step window. All inside

the Credentials module.

#17

Rate Limiting Financial Endpoints with Sliding Windows

Partitioned rate limiters per endpoint sensitivity: 10 login attempts/30s (IP), 5 TOTP verifications/30s (userId), 30 financial transactions/30s (userId). How chaining a sliding window + concurrency limiter covers both burst and parallel abuse.

#18

Request Timestamp Validation — Defending Against Replay Attacks

ApplyDelta validates RequestedAt within a +/-5-minute tolerance. A small guard that closes a window for replayed financial transaction requests.

OBSERVABILITY

#19

Prometheus Metrics in a .NET Domain Layer

Counters and histograms wired directly into the Event Store, Outbox processor, and Account command handlers. What gets measured: event append latency, outbox batch duration, transaction success/failure rates. No APM vendor required.

#20

Centralizing Logs in Grafana Loki with Serilog

Structured JSON logging to file + Loki sink. Every log line carries app and env labels. How rolling file sinks (hourly, 7-day retention) complement Loki for local vs. production debugging.

#21

Health Checks as a First-Class API Citizen

GET /health returns SQL Server connectivity status with duration and per-check detail. Why health endpoints matter even in solo projects: Docker Compose depends on them, and they catch misconfigured connection strings before users do.

INFRASTRUCTURE

#22

Docker Compose as a Full Dev Environment

One docker-compose up spins up SQL Server, Redis, Prometheus, Grafana, and Loki. No cloud dependency, no shared dev database. How I structured the compose file and why port mapping matters for local debugging.

#23

Integration Tests with Testcontainers

SqlFixture spins up a real SQL Server container per test run via Testcontainers.MsSql. Schema initialization, per-test database resets, and why this catches bugs that in-memory mocks never will.

#24

Separating Unit and Integration Tests in One Project

Two namespaces (Unit.* / Integration.*), one test project. The --filter flag on dotnet test lets CI run unit tests without Docker and integration tests separately. How to structure this cleanly in xUnit.

#25

Redis Caching in a CQRS Query Handler

Adding IDistributedCache directly inside the query handler — cache read at the top, write after DB fetch, TTL from config. Why I avoided the decorator pattern and kept cache logic co-located with handler logic.

AWS INTEGRATION

#26

Async Profile Image Uploads with S3 and System.Threading.Channels

User creation and profile updates never wait for image processing. A Channel<T> acts as an in-memory queue: the command handler writes the job and returns immediately. A BackgroundService reads jobs and uploads to S3 asynchronously. Fire-and-forget without losing the request.

#27

Image Validation Before S3 Upload — Magic Bytes, MIME, and Size

Accepting only JPEG, PNG, and WebP images up to 2 MB. Validation checks three things: declared content type against an allowlist, magic byte signatures in the raw binary (not just file extensions), and byte count. Images that fail never reach S3.

#28

DynamoDB as a Metadata Store Alongside SQL Server

SQL Server owns the domain read model; DynamoDB owns image metadata (UserId, ImageId, S3 key, size, created date). Two databases in one system — each chosen for what it does best. How to write DynamoDB records from a .NET BackgroundService with the AWS SDK.

#29

Keeping Event Sourcing Consistent After an Async Side Effect

When the background service confirms the S3 upload, it dispatches an UpdateUserProfileImage command. This raises a UserProfileImageUpdatedEvent, appends it to the Event Store, and lets the Outbox projection write the S3 key back to the User table. The image URL reaches the read model through the same event-sourced pipeline as every other state change.

#30

IAM Least-Privilege for S3 and DynamoDB

One IAM policy: s3:PutObject on the profile images bucket, dynamodb:PutItem on the metadata table. No GetObject, no ListBuckets, no DeleteItem. How to scope AWS credentials to exactly the operations your service performs and nothing more.

#31

S3 Pre-Signed URLs for Secure Profile Image Retrieval

The backend never exposes S3 credentials to clients. GET /user/get/{userId} generates a time-limited pre-signed GET URL (1-minute expiry) on demand using the stored S3 key. After TOTP verification the URL is returned as a plain response-body field alongside the HttpOnly auth cookies — it is deliberately NOT a JWT claim, since a 1-minute URL inside a 30-minute token would be dead weight. Each URL is scoped, short-lived, and requires no client-side AWS credentials. Only s3:GetObject on the exact key is needed.

API DESIGN

#32

API Versioning with URL Segments and Media Types

Asp.Versioning wired with both UrlSegmentApiVersionReader and MediaTypeApiVersionReader. Clients can call /api/v1/accounts or pass api-version=1.1 in the Accept header. Swagger shows both versions automatically.

#33

Designing a Financial Transaction API (ApplyDelta)

A single POST /apply-delta endpoint covers deposits, withdrawals, credit purchases, and payments. How the OperationType enum + validation rules route domain logic without endpoint sprawl.

STRUGGLES & TURNOVERS

#34

The Windows TOTP Clock Skew Disaster

TOTP verification kept failing with a massive verification window set as a workaround — until I found out Windows system time was wrong. Fixed the clock, tightened the window to +/-30 seconds. What I learned about time-sensitive auth and local dev environments.

#35

Eventual Consistency Bites: The Outbox Race Condition

After a command succeeds, the projection update happens asynchronously (~500ms poll interval). A GET immediately after a POST can return stale data. How I document this known gap and what the right fix would be (synchronous read-your-writes option).

#36

UTF-16 LE SQL Script Encoding on Windows

Every time I edited SQL test scripts with PowerShell, line endings were mangled. Lesson: treat SQL migration files as binary artifacts and never let a shell script touch them. How I changed my workflow to avoid this entirely.

#37

Building Event Sourcing Without a Framework

No Marten, no EventStore DB, no NEventStore — just SQL Server and ADO.NET. The reward: deep understanding of versioning, concurrency, and projection consistency. The cost: every edge case is your problem. Was it worth it for a learning project?

#38

The Login That Never Checked the Password

A security refactor stripped the bcrypt hash from the read repository (correct!) — but the verification step silently died with it. For a while, any password logged in; only the email had to exist. Worse: the login response handed back a fresh TOTP QR code, so email alone could complete 2FA. The fix kept the hash out of the read model: the login handler now rebuilds the credentials aggregate from the Event Store and verifies the password inside the domain — the hash never leaves the handler.

#39

Replay-Proofing Apply(): DateTimeOffset.UtcNow Is a Bug

Apply() runs when an event is born AND on every replay — anything reading the clock inside it produces different state per rebuild. My credit-card interest was computed from 'days past due as of NOW', so every aggregate replay grew the debt. Closed dates drifted to the replay day. The rule: inside Apply(), only evt.Timestamp exists; UtcNow belongs at Raise() time, frozen into the event. Bonus lesson: never Raise() inside Apply() — replaying raises phantom events and corrupts the version counter.

CLAUDE CODE & AI-ASSISTED DEVELOPMENT

#40

Using Claude Code as a Pair Programmer on a Solo Project

Claude Code (Anthropic's CLI) as a development partner: generating handlers, writing tests, spotting security issues, and navigating a large codebase. What it does well and where human judgment is still non-negotiable.

#41

Teaching Claude My Coding Standards via Persistent Memory

Claude Code has a file-based memory system that persists across sessions. I trained it on project-specific rules: no decorator pattern for caching, no parallel retrieval methods, skip Docker failures, API versioning scheme. How this eliminates repetitive correction.

#42

AI-Generated Code in a Financial Domain — Where to Trust It

Money math, concurrency, and security are places I verify every line Claude writes. CRUD scaffolding and test boilerplate are places I let it run freely. Where the line is and why it matters more in a financial system than a typical CRUD app.

#43

Prompting for Architecture, Not Just Code

Using Claude Code to think through design decisions — Outbox vs. direct dispatch, decorator vs. inline caching, modular monolith vs. microservices. How to get useful architectural input from an AI without outsourcing the decision.

#44

"Rate My API 0 to 10" — Adversarial Review with an AI

I asked Claude Code to score the whole API. First pass: 6.5 — it found an auth bypass (the password that was never verified), event streams keyed by the wrong identity, and non-deterministic replays. Three review-fix-rescore cycles later: 8.5, with every finding verified against real code. Why 'grade this and tell me why' extracts harsher, more useful reviews than 'find bugs'.

Total: 44 topics across 9 categories.